I²C PROTOCOL

(Inter-Integrated Circuit)

Complete Structured Notes

Covering: Protocol Basics • SDA/SCL Signals • I²C Modes • START/STOP Conditions • Address Phase, ACK & NACK • Data Validity • Master Write/Read • Repeated START (Sr) • STM32F407x Peripheral Driver Overview

Table of Contents

1. I²C Protocol — Introduction & Comparison with SPI

1.1 Pronunciation

- I²C — pronounced 'I-squared-C'
- I2C — pronounced 'I-two-C'

1.2 Definition

I²C (Inter-Integrated Circuit) is a **serial communication protocol** used for communication between integrated circuits (ICs) that are located close to each other on the same PCB.

Unlike SPI, I²C follows a **standardized specification** developed by **NXP Semiconductors**, making communication more reliable and standardized across manufacturers.

1.3 Why I²C is More Complex than SPI

Unlike SPI, I²C defines a full set of rules and mechanisms:

- Data transmission rules
- Data reception rules
- Handshaking mechanism
- Error handling
- Arbitration between masters
- Acknowledgement mechanism
- Clock stretching

■ *SPI = Simple communication protocol | I²C = Standardized communication protocol with many built-in features.*

1.4 Features of I²C

- Serial communication
- Only 2 wires required
- Address-based communication
- Supports multiple masters
- Automatic acknowledgements
- Hardware arbitration
- Clock stretching
- Half-duplex communication

1.5 I²C vs SPI — Comparison Table

Feature	I²C	SPI
Specification	Standard specification by NXP	No universal standard
Number of Wires	2	4 or more
Multi-master	Supported	Not defined in protocol
Arbitration	Automatic (hardware)	Software responsibility
ACK/NACK	Automatic	Not supported
Communication Type	Half Duplex	Full Duplex
Addressing	Uses device addresses	Uses Slave Select pin

Clock Stretching	Supported	Not supported
Maximum Speed	Up to 4 MHz	Much higher (Peripheral Clock / 2)
Data Rate	Low	Very High

1.6 Detailed Point-by-Point Comparison

1.6.1 Specification

I²C: Designed by NXP Semiconductors. A dedicated specification defines data transfer, timing, bus control, error handling, addressing, arbitration, and ACK/NACK — making I²C devices from different manufacturers compatible with each other.

SPI: No universal standard exists. Some companies such as Motorola and Texas Instruments (TI) have their own proprietary specifications.

1.6.2 Multi-Master Capability

I²C: Supports multiple masters on the same bus (e.g., Master 1, Master 2, and multiple slaves). If two masters try to communicate simultaneously, the hardware automatically decides which master gets control — this process is called **Arbitration**, and requires no software intervention.

SPI: The SPI protocol itself does not define multi-master communication. Some microcontrollers (e.g., STM32 SPI peripherals) support multiple masters, but arbitration and conflict handling must be implemented in software.

1.6.3 Automatic Acknowledgement (ACK)

I²C: Every byte received is automatically acknowledged. Hardware automatically generates ACK (Acknowledgement) or NACK (Not Acknowledged) — no software implementation is required.

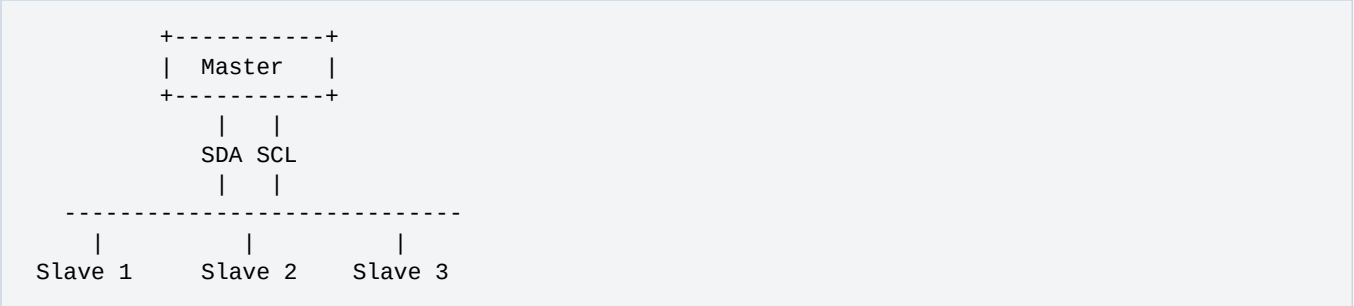
SPI: No acknowledgement mechanism exists. If acknowledgement is needed, the programmer must implement it manually.

1.6.4 Number of Communication Pins

I²C requires only 2 wires:

Pin	Full Form	Function
SDA	Serial Data Line	Carries data
SCL	Serial Clock Line	Carries the clock generated by the master

All devices share the same SDA and SCL lines:



SPI typically requires MOSI, MISO, SCLK, and SS (Slave Select). With multiple slaves, each slave needs its own separate Slave Select line.

Example: 1 Master + 3 Slaves → Master requires MOSI, MISO, SCLK, SS1, SS2, SS3 = **6 pins total**.

Configuration	Pins Required
SPI	6 pins (or more)

I²C	Only 2 pins

■ *Reduced pin count is one of the biggest advantages of I²C over SPI.*

1.6.5 Pull-up Resistors

The SDA and SCL lines are connected to VCC through pull-up resistors:

```
VCC              VCC
 |                |
 R                R
 |                |
 SDA --------     SCL --------
 |                |
Devices          Devices
```

(The full purpose of pull-up resistors is explained in the electrical characteristics section — see Section 2.)

1.6.6 Address-Based Communication

This is one of the biggest differences from SPI. Every slave has its own unique address, and the master communicates by sending that slave's address.

```
Master
   |
Address = 0x20  -> Slave 1
Address = 0x30  -> Slave 2
Address = 0x45  -> Slave 3
```

SPI has no addressing — the master selects slaves using the Slave Select (SS/CS) pin.

1.6.7 Communication Type

I²C is **Half Duplex** — only one direction of communication at a time (Master→Slave OR Slave→Master).

SPI is **Full Duplex** — both devices transmit and receive simultaneously.

1.6.8 Speed

This is one disadvantage of I²C. Maximum speed depends on the I²C mode:

- Ultra Fast Mode maximum: **4 MHz**
- However, many microcontrollers support only 100 kHz, 400 kHz, or 1 MHz
- Example: Most STM32 controllers max out at 400 kHz or 1 MHz

SPI speed = Peripheral Clock ÷ 2. Example: Peripheral Clock = 40 MHz → SPI Speed = 20 MHz — much faster than I²C.

1.6.9 Clock Stretching

A unique feature of I²C. Suppose a slave is busy — instead of losing data, the slave holds the SCL line LOW. The master waits until the slave releases the clock. This mechanism is called **Clock Stretching**.

- Prevents data loss
- Allows slower slaves to communicate safely

SPI: A slave cannot stop the master's clock — no clock stretching exists in SPI.

1.6.10 Data Rate

I²C has a lower data rate than SPI, so it is not suitable for high-speed data transfer.

1.7 Suitable Applications

I²C Applications	SPI Applications
Temperature sensors	LCD displays

EEPROM	SD Cards
RTC	Audio streaming
Accelerometers	Flash memory
Humidity sensors	High-speed ADC/DAC
Pressure sensors	Camera modules
Small displays	
Configuration registers	

1.8 Worked Example: Speed Comparison

Suppose an STM32F4 with Peripheral Clock = 40 MHz:

Protocol	Maximum Data Rate
I²C	400 kbps
SPI	20 Mbps

SPI is 20 Mbps ÷ 0.4 Mbps = **50**. So SPI is approximately **50 times faster** than I²C.

1.9 Important Terminologies

Term	Meaning
Transmitter	Device that sends data onto the bus
Receiver	Device that receives data from the bus
Master	Initiates communication, generates the clock, controls the bus, terminates communication. Only the master starts communication.
Slave	The device addressed by the master. A slave never starts communication on its own.
Multi-Master	A bus where more than one master can communicate
Arbitration	If two masters start communication simultaneously, hardware automatically determines which master continues and which waits; the winning message is transmitted without corruption. Handled automatically by I²C hardware.

1.10 Advantages and Disadvantages of I²C

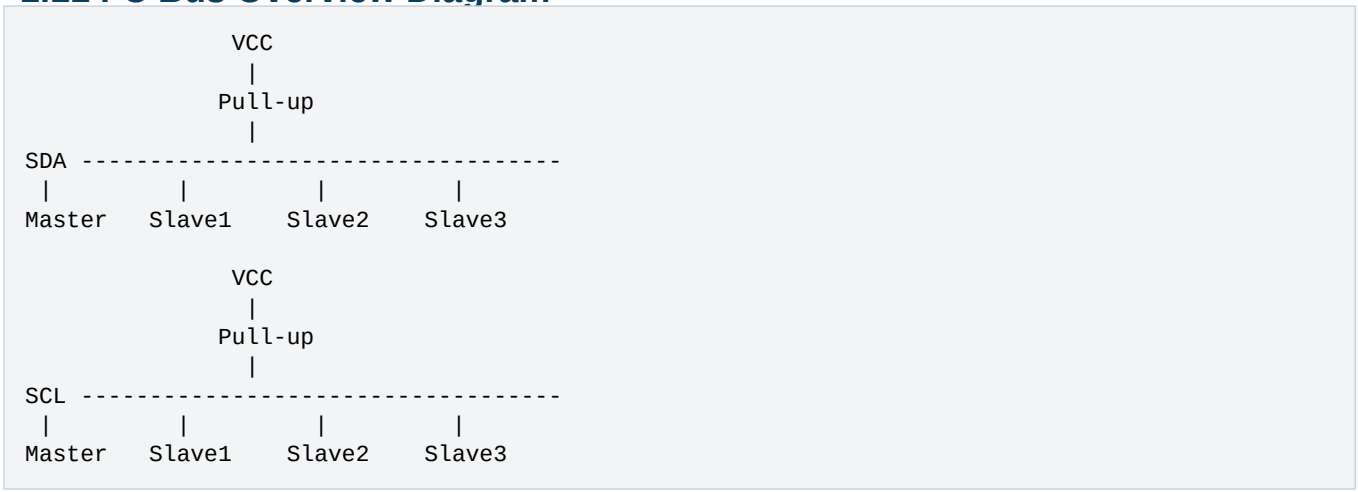
Advantages

- Only two communication wires
- Standardized protocol
- Address-based communication
- Automatic ACK/NACK
- Supports multiple masters
- Hardware arbitration
- Clock stretching
- Easy addition of new devices
- Reduced PCB complexity
- Lower pin usage

Disadvantages

- Slower than SPI
- Half-duplex communication
- Lower data rate
- Not suitable for high-speed streaming
- Performance decreases with long wires and many devices

1.11 I²C Bus Overview Diagram



2. SDA and SCL Signals — Electrical Characteristics

2.1 The Two Lines

Signal	Full Form	Purpose
SDA	Serial Data Line	Transfers data between master and slave
SCL	Serial Clock Line	Carries the clock generated by the master

Both lines are shared by all devices connected to the I²C bus.

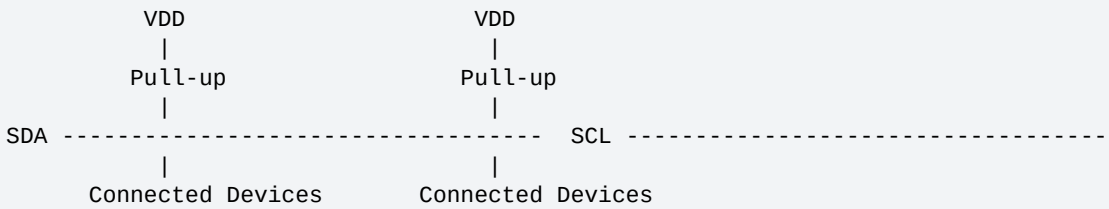
2.2 Important Characteristics of SDA and SCL

- Bidirectional communication lines
- Connected to VDD (positive supply voltage) using pull-up resistors
- Shared among all masters and slaves
- Must use Open Drain (or Open Collector) output configuration
- When the bus is idle, both lines remain HIGH

2.3 Bidirectional Nature

Unlike SPI, where lines have fixed directions (MOSI: Master→Slave, MISO: Slave→Master), in I²C the SDA line can both transmit and receive data, and SCL can also be influenced by both master and slave (during clock stretching). Thus both lines are bidirectional.

2.4 Pull-up Resistors



Purpose of Pull-up Resistors

- Keep the lines at logic HIGH when no device is driving them
- Allow devices to pull the line LOW without causing conflicts
- Enable multiple devices to safely share the same bus

2.5 Bus Idle Condition

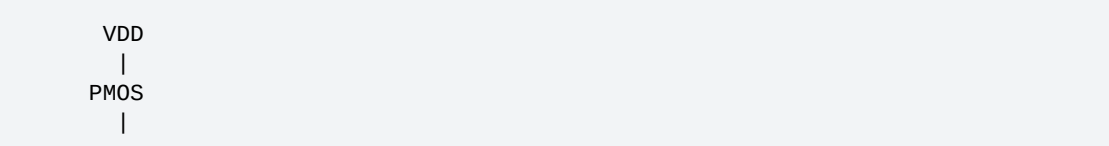
When no communication is taking place: **SDA = HIGH** and **SCL = HIGH**. This condition is called the **Idle Bus State**.

2.6 Output Configuration Requirement

■ GOLDEN RULE: All I²C output pins must be configured as Open Drain (or Open Collector). This is mandatory for proper I²C operation.

2.6.1 Push-Pull vs Open Drain

Normally, a GPIO output uses **Push-Pull** configuration:



```
Output Pin
      |
    NMOS
      |
     GND
```

- PMOS drives the pin HIGH
- NMOS drives the pin LOW

In **Open Drain** configuration, the PMOS is removed and only the NMOS remains:

```
Output Pin
     |
   NMOS
     |
    GND
```

- The pin can actively pull the line LOW
- It **cannot** actively drive the line HIGH
- A pull-up resistor is required to obtain the HIGH state

2.6.2 Why Open Drain is Used

- Allows multiple devices to share the same communication lines
- Any device can pull the bus LOW
- No electrical conflict occurs if multiple devices operate simultaneously

Without Open Drain, two devices could drive opposite logic levels at the same time, causing **bus contention** and possible hardware damage.

2.7 GPIO Configuration for I²C

Whenever a GPIO pin is used for I²C communication:

- Configure it as Alternate Function
- Set output type to Open Drain
- Enable either Internal Pull-up or External Pull-up resistor

Setting	Value
Mode	Alternate Function
Output Type	Open Drain
Pull-up	Internal or External
Speed	As required

2.8 Pull-up Options

Type	Advantages	Disadvantages
Internal Pull-up	No external components; simple design	Usually weak; not suitable for high-speed communication
External Pull-up	Stronger pull-up; better signal quality; preferred in practical designs	Requires an external resistor component

2.9 Pull-up Resistor Value

The pull-up resistor value cannot be chosen arbitrarily. It depends on:

- Bus capacitance
- Supply voltage
- Communication speed
- I²C timing requirements

The correct value is calculated using formulas defined in the I²C specification.

2.10 Bus Capacitance

Each device connected to the I²C bus adds some capacitance:

```
Master
   |
---------
|   |   |
S1  S2  S3
```

As more devices are connected:

- Total bus capacitance increases
- Rise time becomes slower
- Communication speed may reduce
- Maximum number of devices becomes limited

■ *Bus capacitance limits the number of interfaces that can be connected to the I²C bus.*

2.11 SDA and SCL Configuration Summary

SDA	SCL
Open Drain	Open Drain
Pull-up enabled	Pull-up enabled
Bidirectional	Bidirectional
—	Used for clock generation

2.12 Troubleshooting Tip

■ *Step 1: Initialize the I²C peripheral. Step 2: Measure the voltage between SDA-Ground and SCL-Ground. Expected: SDA ≈ VDD and SCL ≈ VDD (e.g., for a 3.3 V system, SDA ≈ 3.3 V, SCL ≈ 3.3 V).*

If either line is not HIGH, possible causes include:

- Missing pull-up resistor
- Incorrect GPIO configuration
- Push-Pull mode instead of Open Drain
- Faulty wiring
- Short circuit
- Device holding the bus LOW

2.13 Common I²C Configuration Mistakes

Mistake	Result
Configuring SDA/SCL as Push-Pull	Bus contention; communication failure
Forgetting pull-up resistors	Lines never go HIGH; bus remains stuck; no communication
Incorrect pull-up value	Slow rise time; timing violations; communication errors

Wrong GPIO mode	I²C peripheral cannot control the pins correctly

2.14 Practical Checklist Before Debugging

- GPIO mode is set to Alternate Function
- Output type is Open Drain
- Pull-up resistor is present (internal or external)
- SDA is HIGH when idle
- SCL is HIGH when idle
- Supply voltage is correct (e.g., 3.3 V)
- Wiring is correct
- No device is permanently pulling the bus LOW

2.15 Key Takeaways

- I²C uses two bidirectional lines: SDA (Serial Data) and SCL (Serial Clock)
- Both lines are connected to VDD through pull-up resistors
- When the bus is idle, both SDA and SCL remain HIGH
- All I²C pins must be configured as Open Drain (or Open Collector)
- Open Drain allows multiple devices to safely share the same bus
- Pull-up resistors are essential because Open Drain outputs cannot drive the line HIGH
- Pull-up resistors may be internal or external; external resistors are commonly preferred
- Bus capacitance affects the maximum number of connected devices and communication speed
- A key troubleshooting step is checking that SDA and SCL are at VDD after I²C initialization

3. I²C Communication Modes

The I²C protocol supports multiple communication modes, each providing a different maximum data transfer rate. The mode selected determines maximum communication speed, compatibility with other devices, timing configuration, and driver configuration.

3.1 Types of I²C Modes

Mode	Max Data Rate	STM32F4 Support	Typical Usage
Standard Mode (Sm)	100 kbps	Supported by almost all STM32 MCUs	Basic sensors, RTCs, EEPROMs
Fast Mode (Fm)	400 kbps	Supported by almost all STM32F4 MCUs	Faster sensors, displays
Fast Mode Plus (Fm+)	1 Mbps	Supported only by some STM32F4 MCUs	High-performance peripherals
High-Speed Mode (Hs)	3.4 Mbps	Not supported by STM32F4 family	Very high-speed I²C communication

■ *Support for Fast Mode Plus and High-Speed Mode depends on the specific microcontroller. Always check the Reference Manual (RM) and datasheet.*

3.2 Overview

```
Standard Mode      -> 100 kbps
Fast Mode          -> 400 kbps
Fast Mode Plus     -> 1 Mbps
High-Speed Mode    -> 3.4 Mbps
```

As the mode changes, the maximum allowable communication speed increases.

3.3 Standard Mode (Sm) — 100 kbps

- First mode introduced in the original I²C specification
- Supported by almost every I²C-enabled microcontroller
- Used by most low-speed peripherals
- Suitable for applications that do not require high data throughput

Common Applications

- Real-Time Clock (RTC)
- Temperature sensors
- Humidity sensors
- EEPROM
- Pressure sensors
- Basic environmental sensors

Compatibility Rule

■ **GOLDEN RULE: Standard Mode devices are NOT upward compatible — a device that supports only Standard Mode cannot reliably communicate on a bus running at Fast Mode (400 kbps) or higher.**

Example: A sensor supports only 100 kbps but the master is configured for 400 kbps → communication fails because the sensor cannot keep up with the faster clock.

3.4 Fast Mode (Fm) — 400 kbps

- Approximately 4× faster than Standard Mode
- Supported by almost all STM32F4 microcontrollers
- Suitable for peripherals that require faster communication

Compatibility

Fast Mode devices are **downward compatible** — a Fast Mode device can also operate on a Standard Mode (100 kbps) bus; communication simply takes place at the lower speed.

Example: Fast Mode master + Standard Mode bus → Operates at 100 kbps.

Important Limitation

Although Fast Mode devices can operate at 100 kbps, Standard Mode-only devices must **not** be placed on a Fast Mode (400 kbps) bus. If the bus runs at 400 kbps, Standard Mode devices cannot respond correctly and communication becomes unreliable, potentially causing the bus to enter unpredictable states.

3.5 Fast Mode Plus (Fm+) — 1 Mbps

- Faster than Standard Mode and Fast Mode
- Supported only by selected STM32F4 microcontrollers
- Requires hardware support — check the Reference Manual before use

3.6 High-Speed Mode (Hs) — 3.4 Mbps

- Designed for very high-speed I²C communication
- Not supported by the STM32F4 family
- Support in other STM32 families (such as F3 or F7) depends on the specific device — always verify hardware support

3.7 Comparison of All Modes

Feature	Standard	Fast	Fast+	High Speed
Maximum Speed	100 kbps	400 kbps	1 Mbps	3.4 Mbps
Original I²C Mode	Yes	No	No	No
Supported by Most STM32F4 MCUs	Yes	Yes	Partial	No
Commonly Used	Yes	Yes	Limited	Rare
Covered in This Course	Yes	Yes	No	No

3.8 Compatibility Summary

```
Standard Mode Device
        |
        +-- Can communicate at 100 kbps : YES
        +-- Cannot communicate at 400 kbps or above : NO

Fast Mode Device
        |
        +-- Can communicate at 400 kbps : YES
        +-- Can also operate at 100 kbps : YES
```

Device Type	100 kbps Bus	400 kbps Bus

Standard Mode Device	Works	Not Supported
Fast Mode Device	Works	Works

3.9 Driver Development Consideration

When writing an I²C driver, the communication mode should be **configurable**. The driver should allow the user to select the desired operating mode (e.g., Standard Mode 100 kbps or Fast Mode 400 kbps), and the driver must then configure the I²C peripheral registers accordingly.

■ *The course and driver-development exercises use only Standard Mode (100 kbps) and Fast Mode (400 kbps). Fast Mode Plus (1 Mbps) and High-Speed Mode (3.4 Mbps) are not covered.*

3.10 Practical Selection Guidelines

Application	Recommended Mode
RTC	Standard Mode
Temperature Sensor	Standard Mode
EEPROM	Standard Mode
Humidity Sensor	Standard Mode
Moderate-speed sensors	Fast Mode
Faster displays/peripherals	Fast Mode
High-performance I²C devices	Fast Mode Plus (if supported)

3.11 Advantages / Limitations of Higher Modes

Advantages

- Faster data transfer
- Reduced communication time
- Better performance for high-throughput devices

Limitations

- Not supported by all microcontrollers
- May require stronger pull-up resistors and stricter timing
- Older Standard Mode-only devices cannot operate on higher-speed buses

3.12 Key Takeaways

- I²C defines four communication modes: Standard (100 kbps), Fast (400 kbps), Fast Plus (1 Mbps), High-Speed (3.4 Mbps)
- Standard Mode was the first I²C mode introduced and is supported by nearly all devices
- Fast Mode is about 4× faster than Standard Mode and is widely supported on STM32F4 microcontrollers
- Fast Mode Plus is available only on selected microcontrollers and requires hardware support
- High-Speed Mode (3.4 Mbps) is not supported by the STM32F4 family
- Standard Mode-only devices are not upward compatible and should not be used on buses running at 400 kbps or higher
- Fast Mode devices are downward compatible and can communicate on Standard Mode buses at 100 kbps

- The communication mode should be a configurable driver parameter for correct timing/speed initialization

4. I²C Protocol Basics — Communication Sequence

The I²C protocol follows a well-defined sequence for communication between a master and a slave. A complete I²C communication consists of:

- START condition
- Address Phase
- ACK/NACK
- Data Transfer
- ACK/NACK after each byte
- STOP condition

4.1 Basic Communication Flow

```
Master
   |
   v
START
   |
Address + R/W
   |
ACK
   |
Data Byte
   |
ACK
   |
Data Byte
   |
ACK
   |
STOP
```

■ GOLDEN RULE: The master always initiates communication. The slave never starts communication.

The master: starts communication, generates the clock, selects the slave, and ends communication.

4.2 Step 1 — START Condition

Communication always begins with the START condition, generated by the master.

Purpose

- Indicates that communication is beginning
- Takes control of the I²C bus
- Notifies all connected devices to listen

```
Master
   |
Generate START
   |
Bus becomes busy
```

4.3 Step 2 — Address Phase

Immediately after the START condition comes the Address Phase, which is always 8 bits (1 byte):

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
A6	A5	A4	A3	A2	A1	A0	R/W

- First 7 bits → Slave Address
- Last bit → Read/Write (R/W) bit

Example slave address: 1010010. Every slave connected to the bus has its own unique address; the addressed slave compares the received address with its own.

Read/Write (R/W) Bit

R/W Bit	Meaning
0	Master writes data to the slave
1	Master reads data from the slave

■ *Memory Tip — 0 = Write to Slave, 1 = Read from Slave.*

■ **GOLDEN RULE: Every transmission on the SDA line must be exactly 8 bits (1 byte). You cannot send 7, 9, or 16 bits — each transfer is byte-oriented.**

4.4 ACK (Acknowledgement)

After every byte transmitted, an ACK (Acknowledgement) bit follows.

- Confirms that the previous byte was successfully received
- Synchronizes communication between master and slave

■ *Rule: Every 8-bit byte is followed by one ACK/NACK bit.*

4.5 Write Operation (R/W = 0)

```
START
   |
Address + Write (0)
   |
Slave ACK
   |
Data Byte
   |
Slave ACK
   |
Data Byte
   |
Slave ACK
   |
STOP
```

Steps: (1) Master generates START. (2) Master sends 7-bit slave address + Write bit (0). (3) Addressed slave checks the address. (4) If it matches, the slave sends ACK. (5) Master sends one data byte. (6) Slave acknowledges the received byte. Steps 5-6 repeat for additional bytes. (7) Master generates STOP to end communication.

4.6 Read Operation (R/W = 1)

```
START
   |
Address + Read (1)
   |
Slave ACK
   |
Slave sends Data
   |
Master ACK
   |
Slave sends Data
```

```
   |
Master ACK
   |
STOP
```

Steps: (1) Master generates START. (2) Master sends 7-bit slave address + Read bit (1). (3) Slave checks the address and sends ACK. (4) Slave transmits one data byte. (5) Master acknowledges the received byte. Additional bytes can be transferred with ACK after each byte. (6) Master generates STOP when it has received enough data.

4.7 ACK Responsibility

Communication	Who Sends ACK?
Address sent by Master	Slave
Data written by Master	Slave
Data sent by Slave (Read Operation)	Master

4.8 Data Transfer Order

I²C transmits data **Most Significant Bit (MSB) first**.

Example: Data = 11001010 → Transmission order: 1 → 1 → 0 → 0 → 1 → 0 → 1 → 0 (MSB sent first).

4.9 Multiple Data Bytes

The master may send or receive multiple bytes. Communication continues until the master decides to stop.

Write Sequence	Read Sequence
Address / ACK	Address / ACK
Byte 1 / ACK	Byte 1 / ACK
Byte 2 / ACK	Byte 2 / ACK
Byte 3 / ACK	Byte 3 / ACK
STOP	STOP

4.10 STOP Condition

Communication ends with the STOP condition, generated by the master.

- Releases the I²C bus
- Indicates that communication has finished
- Allows another master (in a multi-master system) to use the bus

```
Master
   |
Generate STOP
   |
Bus becomes free
```

4.11 Bus Ownership

Event	Bus State
After START	Bus Busy — no other master can communicate until the current transaction finishes (subject to arbitration)

After STOP	Bus Free — another master may now initiate communication

4.12 Complete Timing Overviews

Write

```
Master
  |
START
  |
Address + W
  |
Slave ACK
  |
Data Byte
  |
Slave ACK
  |
Data Byte
  |
Slave ACK
  |
STOP
```

Read

```
Master
  |
START
  |
Address + R
  |
Slave ACK
  |
Slave Data
  |
Master ACK
  |
Slave Data
  |
Master ACK
  |
STOP
```

4.13 Key Takeaways

- Only the master initiates and terminates I²C communication
- Every I²C transaction begins with a START condition and ends with a STOP condition
- The first byte after START is the Address Phase: 7-bit slave address + 1-bit R/W flag
- R/W = 0 indicates a Write operation (Master → Slave); R/W = 1 indicates a Read operation (Slave → Master)
- All transfers are byte-oriented: each byte is 8 bits, followed by an ACK/NACK bit
- During a Write, the slave acknowledges the address and each data byte
- During a Read, the master acknowledges each data byte received from the slave
- Data is transmitted Most Significant Bit (MSB) first
- The STOP condition releases the bus, allowing another master to begin communication

5. START and STOP Conditions

Every I²C communication (transaction) is bounded by two special conditions generated by the master: **START Condition (S)** and **STOP Condition (P)**.

5.1 Transaction Sequence

```
START (S)
     |
Address Phase
     |
ACK
     |
Data Transfer
     |
ACK(s)
     |
STOP (P)
```

■ **GOLDEN RULE: Every I²C transaction begins with a START condition and ends with a STOP condition. START is represented by S, STOP by P.**

5.2 START Condition (S)

Definition: A START condition is generated when **SDA transitions from HIGH to LOW while SCL remains HIGH**.

```
SCL --------------- HIGH -----------------

SDA --------+
            |
            +-------- LOW

     HIGH -> LOW while SCL = HIGH
            = START (S)
```

Purpose of START

- Begins an I²C transaction
- Indicates that the bus is now busy
- Gives the master control of the bus
- Alerts all slave devices to listen for the upcoming address

5.3 STOP Condition (P)

Definition: A STOP condition is generated when **SDA transitions from LOW to HIGH while SCL remains HIGH**.

```
SCL --------------- HIGH -----------------

SDA ----- LOW ------+
                    |
                    +-------- HIGH

     LOW -> HIGH while SCL = HIGH
           = STOP (P)
```

Purpose of STOP

- Ends the current I²C transaction
- Releases the bus
- Returns SDA and SCL to their idle HIGH state (via pull-up resistors)

- Allows another master (in a multi-master system) to begin communication

5.4 Bus States

State	Condition
Idle Bus	No communication taking place: SDA = HIGH, SCL = HIGH. The bus is free.
Busy Bus	After a START condition, the current master owns the bus until it issues a STOP (or arbitration changes ownership).
Free Bus	After a STOP condition, the master releases the bus and another master may initiate communication.

5.5 START vs STOP Summary

Feature	START (S)	STOP (P)
SDA Transition	HIGH -> LOW	LOW -> HIGH
SCL State	HIGH	HIGH
Generated By	Master	Master
Purpose	Begin communication	End communication
Bus State	Busy	Free

■ *Interview Tip: A very common interview question is to define the START and STOP conditions. Remember the SDA transition direction while SCL is HIGH.*

5.6 Address Phase Review

Immediately after START, the master transmits the Address Phase (1 byte / 8 bits):

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
A6	A5	A4	A3	A2	A1	A0	R/W

- First 7 bits: Slave address
- 8th bit: Read/Write (R/W)

```
Clock:  1 2 3 4 5 6 7 8 9
         | | | | | | | | |
Data :  A A A A A A A R ACK
```

- Clocks 1-7: Slave address
- Clock 8: R/W bit
- Clock 9: ACK/NACK from the receiver

5.7 ACK Timing per Phase

Address Phase

```
Address (8 bits)
        |
        v
ACK (9th clock)
```

Data Phase (Write)

```
Master Data (8 bits)
          |
          v
```

```
Slave ACK (9th clock)
```

Data Phase (Read)

```
Slave Data (8 bits)
        |
        v
Master ACK (9th clock)
```

5.8 Repeated START (Introductory Overview)

Sometimes the master needs to continue communication without releasing the bus. Instead of generating a STOP followed by a new START, it generates another START directly — this is a **Repeated START**.

Definition: A Repeated START is a START condition generated without first issuing a STOP condition.

```
START
  |
Address
  |
ACK
  |
Data
  |
ACK
  |
Repeated START
  |
Address
  |
ACK
  |
Read/Write Continues
```

Characteristics

- Generated by the master
- No STOP occurs before it
- The bus remains busy
- Bus ownership is retained by the same master

Repeated START is commonly used when the master first writes a register address to a slave and then immediately reads data from that register — the direction changes (Write→Read) without releasing the bus. (Covered in full detail in Section 9.)

5.9 Bus State During Repeated START

```
START
  |
Bus Busy
  |
Repeated START
  |
Bus Still Busy
  |
STOP
  |
Bus Free
```

■ *Unlike a STOP, a Repeated START does not release the bus.*

5.10 Automatic Master/Slave Role in Microcontrollers

Many microcontroller I²C peripherals support both Master and Slave modes. Typically, you do not manually configure the mode:

- When the peripheral generates a START, it automatically operates as a Master
- When it generates a STOP, it releases the bus and can return to a Slave role if required

Many dedicated peripherals (e.g., sensors, RTCs, EEPROMs) are slave-only devices and cannot become masters.

5.11 Common Interview Questions

Q1. What defines a START condition in I²C?

Answer: SDA transitions HIGH → LOW while SCL is HIGH.

Q2. What defines a STOP condition in I²C?

Answer: SDA transitions LOW → HIGH while SCL is HIGH.

Q3. Who generates START and STOP conditions?

Answer: The master.

Q4. When is the I²C bus considered busy?

Answer: Immediately after the START condition.

Q5. When is the I²C bus considered free?

Answer: After the STOP condition (following the required bus-free time).

Q6. What is a Repeated START?

Answer: A new START generated without a preceding STOP, allowing the master to continue communication while retaining control of the bus.

5.12 Key Takeaways

- Every I²C transaction starts with START (S) and ends with STOP (P)
- START: SDA goes HIGH → LOW while SCL is HIGH
- STOP: SDA goes LOW → HIGH while SCL is HIGH
- Both START and STOP are always generated by the master
- The bus is busy after START and free after STOP
- The Address Phase immediately follows START: 7-bit slave address + 1-bit R/W flag
- Every transmitted byte is followed by an ACK/NACK bit on the 9th clock pulse
- A Repeated START allows the master to continue communication without releasing the bus
- Most microcontroller I²C peripherals automatically assume Master role on START and relinquish the bus after STOP

6. Address Phase, ACK and NACK

Immediately after the START condition, the master transmits the Address Phase — always 1 byte (8 bits) long — identifying which slave should communicate and whether the operation is Read or Write.

6.1 Address Phase Structure

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
A6	A5	A4	A3	A2	A1	A0	R/W

- Bits 7-1 (first 7 bits) → Slave Address
- Bit 0 (8th bit) → Read/Write (R/W) bit

6.2 Address Phase Timing

```
Clock : 1 2 3 4 5 6 7 8
        | | | | | | | |
Data  : A A A A A A A R/W
```

- Clock 1-7: Slave Address
- Clock 8: Read/Write bit

6.3 Slave Address

Every I²C slave has a unique 7-bit address. The master sends this address after the START condition. Each slave compares the received address with its own:

- Match found → Responds with ACK
- No match → Ignores the transmission

6.4 Read/Write (R/W) Bit

R/W Bit	Meaning	Data Direction
0	Write Operation	Master → Slave
1	Read Operation	Slave → Master

Write Operation = Address + 0 → Master sends data to the slave.

Read Operation = Address + 1 → Master requests data from the slave.

6.5 Acknowledgement (ACK)

After every transmitted byte, an ACK/NACK bit is transmitted — occupying the **9th clock pulse**.

```
Clock : 1 2 3 4 5 6 7 8 9
        | | | | | | | | |
Data  : A A A A A A A R ACK
```

The 9th clock is reserved exclusively for ACK/NACK.

6.6 ACK Mechanism

The transmitter releases the SDA line during the 9th clock pulse. The receiver then controls the SDA line:

- Pulls SDA LOW → ACK
- Leaves SDA HIGH → NACK

This allows the receiver to indicate whether it successfully received the previous byte.

6.7 ACK / NACK Definitions

■ GOLDEN RULE: ACK: SDA is LOW during the HIGH period of the 9th clock pulse.

```
SCL : ____|~~~|____

SDA : __________
              |
              LOW
              ACK
```

■ GOLDEN RULE: NACK: SDA remains HIGH during the HIGH period of the 9th clock pulse.

```
SCL : ____|~~~|____

SDA : ~~~~~~~~~~~
              NACK
```

6.8 ACK vs NACK Table

SDA during 9th Clock	Meaning
LOW	ACK (Acknowledge)
HIGH	NACK (Not Acknowledge)

6.9 Why ACK is Important

The ACK bit tells the transmitter: the receiver successfully received the byte, communication can continue, and the next byte may be transmitted. Without ACK, the transmitter knows something went wrong.

6.10 ACK After Every Byte

An ACK/NACK occurs after every 8-bit byte, not just after the Address Phase:

```
START
 |
Address
 |
ACK
 |
Data Byte
 |
ACK
 |
Data Byte
 |
ACK
 |
STOP
```

6.11 Who Generates the Clock / Who Sends ACK?

Even during the ACK cycle, the master generates the clock pulses, including the 9th clock pulse. The receiver only controls the SDA line during this pulse.

Transmitted Byte	Receiver	ACK Sent By
Address (from Master)	Slave	Slave
Data (Master → Slave)	Slave	Slave
Data (Slave → Master)	Master	Master

6.12 Write / Read Operation Examples

Write Operation

```
START
 |
Address + W
 |
Slave ACK
 |
Data Byte
 |
Slave ACK
 |
Data Byte
 |
Slave ACK
 |
STOP
```

The slave acknowledges both the address and every data byte.

Read Operation

```
START
 |
Address + R
 |
Slave ACK
 |
Data Byte
 |
Master ACK
 |
Data Byte
 |
Master ACK
 |
STOP
```

The master acknowledges every data byte received from the slave.

6.13 NACK Behavior

A NACK indicates that the receiver did not acknowledge the previous byte. Common reasons include:

- No slave responded to the transmitted address
- Receiver is unable or unwilling to accept more data
- Communication error occurred
- Master has received all the data it needs (during a read operation)

6.14 What Happens After a NACK?

After detecting a NACK, the master can:

- Generate a STOP condition to terminate the transaction
- Generate a Repeated START condition to begin a new transaction without releasing the bus

6.15 Complete Timing Example

```
Clock : 1 2 3 4 5 6 7 8 9

Data  : A A A A A A A R ACK
```

- Bits 1-7: Slave Address

- Bit 8: Read/Write
- Bit 9: ACK or NACK

6.16 Quick Revision Table

Item	Description
Address Phase Length	8 bits (1 byte)
Slave Address	First 7 bits
R/W Bit	8th bit
R/W = 0	Write to Slave
R/W = 1	Read from Slave
ACK/NACK Position	9th clock pulse
ACK	SDA = LOW
NACK	SDA = HIGH
ACK Generated By	Receiver
Clock During ACK	Generated by Master
After NACK	STOP or Repeated START

6.17 Key Rules to Remember

- The Address Phase always follows the START condition
- The Address Phase is always 8 bits: 7-bit slave address + 1-bit R/W flag
- R/W = 0 → Write (Master → Slave); R/W = 1 → Read (Slave → Master)
- Every 8-bit byte is followed by an ACK/NACK on the 9th clock pulse
- During the ACK cycle, the master generates the clock while the receiver controls the SDA line
- ACK = SDA LOW during the HIGH period of the 9th clock; NACK = SDA HIGH during that period
- After a NACK, the master may either issue a STOP condition or a Repeated START condition

7. I²C Data Validity

Data validity in the I²C protocol means that the data present on the SDA (Serial Data) line must remain stable whenever the SCL (Serial Clock) line is HIGH. The receiver samples (reads) the data only while SCL is HIGH.

7.1 Golden Rule of I²C

■ **GOLDEN RULE: SDA can change only when SCL is LOW. (Equivalently: Data must be stable when SCL is HIGH.)**

7.2 Why is This Rule Necessary?

The receiver (master or slave) reads the data during the HIGH period of the clock. If the data changed while the clock was HIGH, the receiver wouldn't know whether the bit is 0, 1, or changing between them — this could cause incorrect data reception.

■ *Clock HIGH → Read Data. Clock LOW → Change Data.*

7.3 Data Validity Timing

```
SCL :   ___|~~~|___|~~~|___|~~~|___

SDA : __0_______1_______0_______1__

        ^       ^       ^       ^
     Data is stable while SCL is HIGH
```

During every HIGH pulse of SCL, SDA remains constant and the receiver safely reads the bit.

7.4 When Can SDA Change?

The SDA line is allowed to change only during the LOW period of SCL:

```
SCL :   ___|~~~|_____|~~~|_____

SDA : __0___/~~~~~~~\___0_______

        ^
     Data changes here
     (SCL is LOW)
```

7.5 Allowed Data Transitions

Transitions 0→1 and 1→0 are allowed only when SCL = LOW:

```
Clock (SCL)
LOW ---------------- HIGH ---------------- LOW

Data (SDA)
0 ------------- stays 0 ------------ changes to 1
```

7.6 What Happens When SCL is HIGH?

When SCL becomes HIGH, the SDA line must remain unchanged:

```
SCL :  ____|~~~~~|____

SDA : _________0________
           Stable
```

7.7 Important Exceptions

There are two exceptions to the data validity rule — during these, SDA is allowed to change even when SCL is HIGH:

- Start Condition
- Stop Condition

Start Condition

Occurs when SCL = HIGH and SDA changes from HIGH → LOW:

```
SCL :  ~~~~~~~~~

SDA :  ~~~~\____
          v
       START
```

Stop Condition

Occurs when SCL = HIGH and SDA changes from LOW → HIGH:

```
SCL :  ~~~~~~~~~

SDA :  ____/~~~
         ^
       STOP
```

7.8 How Does I²C Differentiate Data from Control Signals?

Since data changes only while SCL is LOW, and START/STOP occur while SCL is HIGH, the receiver can easily distinguish between them:

SDA Transition	SCL State	Meaning
Changes	LOW	Data Bit
HIGH → LOW	HIGH	START Condition
LOW → HIGH	HIGH	STOP Condition

■ *This simple rule makes I²C communication very reliable.*

7.9 Summary Diagram

```
          DATA VALIDITY

SCL
      ___       ___       ___
_____|   |_____|   |_____|   |____

SDA
____0_______1_______0_______1______

     ^       ^       ^       ^
 Data remains stable when SCL is HIGH


Data transitions occur here

SCL
____      ______      ______
    |____|      |____|

SDA
____0__/~~~~\___0__/~~~~\____

      ^          ^
   Data changes only
```

```
when SCL is LOW
```

7.10 Key Concept

The receiver samples the data only when the clock is HIGH. Therefore: Clock HIGH → Data must be stable; Clock LOW → Data may change.

7.11 Interview Questions

Q. Why is data allowed to change only when SCL is LOW?

Answer: Because the receiver reads the data during the HIGH period of SCL. Keeping the data stable while SCL is HIGH ensures reliable sampling and prevents incorrect bit detection.

Q. Why are START and STOP conditions generated when SCL is HIGH?

Answer: If SDA changes while SCL is HIGH, it cannot represent normal data (since data is not allowed to change then). The receiver therefore interprets SDA HIGH→LOW (while SCL HIGH) as a START and SDA LOW→HIGH (while SCL HIGH) as a STOP — clearly differentiating control signals from data bits.

7.12 Memory Trick

■ *"High = Hold, Low = Change" — SCL HIGH → Hold the data (stable); SCL LOW → Change the data if needed.*

7.13 Exam & Revision Points

- Data is transmitted on the SDA line; clock is generated on the SCL line
- Data must remain stable while SCL is HIGH
- SDA changes only when SCL is LOW
- Exceptions: START and STOP conditions
- START: SDA HIGH → LOW while SCL HIGH
- STOP: SDA LOW → HIGH while SCL HIGH
- The receiver samples data during the HIGH period of SCL
- This waveform rule is one of the most frequently asked I²C interview questions

■ *One-Line Revision: In I²C, data changes only when SCL is LOW, remains stable when SCL is HIGH, and only START/STOP conditions allow SDA transitions while SCL is HIGH.*

8. Data Transfer — Master Write & Master Read

After understanding the START condition, Address Phase, and Data Validity, the next step is understanding how data is actually transferred between the Master and the Slave. There are two types of communication: Master Writes Data to Slave, and Master Reads Data from Slave.

8.1 Master Writing Data to Slave (R/W = 0)

Master is the transmitter; Slave is the receiver.

Sequence of Operation

- Step 1: Master generates START condition — informs all devices that communication is beginning
- Step 2: Master sends the 7-bit slave address — every slave compares this with its own; only the addressed slave responds
- Step 3: Master sends the R/W bit = 0, meaning master wants to WRITE data to the slave
- Step 4: Slave sends ACK — pulls SDA LOW during the ACK clock pulse, confirming address received correctly and slave ready to receive data
- Step 5: Master sends first data byte (8 bits) on the SDA line
- Step 6: Slave sends ACK — tells master 'I received the byte successfully, send the next byte'
- Step 7: Master sends second byte
- Step 8: Slave sends ACK
- Step 9: Master sends third byte
- Step 10: Slave sends ACK
- Step 11: Master generates STOP condition — ends communication

Master Write Timing Diagram

```
Master                          Slave

START
  |
  |-- 7-bit Address ------------>
  |-- R/W = 0 (Write) ---------->
  <----------- ACK ---------------
  |-- Data Byte 1 --------------->
  <----------- ACK ---------------
  |-- Data Byte 2 --------------->
  <----------- ACK ---------------
  |-- Data Byte 3 --------------->
  <----------- ACK ---------------
STOP
```

Communication Flow Table

Step	Sender	Receiver	Data
1	Master	All Slaves	START
2	Master	Slave	7-bit Address
3	Master	Slave	R/W = 0
4	Slave	Master	ACK
5	Master	Slave	Data Byte 1
6	Slave	Master	ACK
7	Master	Slave	Data Byte 2
8	Slave	Master	ACK

9	Master	Slave	Data Byte 3
10	Slave	Master	ACK
11	Master	All	STOP

■ *During a Write operation: the master transmits data and the slave acknowledges every received byte.*

8.2 Master Reading Data from Slave (R/W = 1)

Slave becomes the transmitter; Master becomes the receiver.

Sequence of Operation

- Step 1: Master generates START condition
- Step 2: Master sends 7-bit slave address
- Step 3: Master sends R/W bit = 1, meaning master wants to READ data from the slave
- Step 4: Slave sends ACK, acknowledging it has been addressed
- Step 5: Slave becomes the transmitter and places the first data byte on SDA; master receives this byte
- Step 6: Master sends ACK, meaning 'I received this byte successfully, send another byte'
- Step 7: Slave sends second byte
- Step 8: Master sends ACK, again indicating continue sending data
- Step 9: Slave sends third byte
- Step 10: Master sends NACK instead of ACK, meaning 'I don't need any more data'; this tells the slave to stop transmitting
- Step 11: Master generates STOP condition, terminating communication

Master Read Timing Diagram

```
Master                          Slave

START
  |
  |-- 7-bit Address ------------->
  |-- R/W = 1 (Read) ------------>
  <----------- ACK ----------------
  <--------- Data Byte 1 ----------
  |------------ ACK -------------->
  <--------- Data Byte 2 ----------
  |------------ ACK -------------->
  <--------- Data Byte 3 ----------
  |----------- NACK --------------->
STOP
```

Communication Flow Table

Step	Sender	Receiver	Data
1	Master	All Slaves	START
2	Master	Slave	7-bit Address
3	Master	Slave	R/W = 1
4	Slave	Master	ACK
5	Slave	Master	Data Byte 1
6	Master	Slave	ACK
7	Slave	Master	Data Byte 2
8	Master	Slave	ACK

9	Slave	Master	Data Byte 3
10	Master	Slave	NACK
11	Master	All	STOP

8.3 Why Does the Master Send ACK While Reading?

During a read operation, the slave does not know how many bytes the master wants. Therefore: ACK = 'Continue sending more bytes'; NACK = 'I have received enough data, stop sending.'

Master Response	Meaning
ACK	Send another byte
NACK	Stop transmitting

8.4 Difference Between Write and Read Operations

Feature	Master Write	Master Read
R/W Bit	0	1
Data Sender	Master	Slave
Data Receiver	Slave	Master
Who sends ACK after each data byte?	Slave	Master
Communication Ends	Master sends STOP	Master sends NACK followed by STOP

8.5 Complete Comparison

MASTER WRITE	MASTER READ
START	START
Address	Address
R/W = 0	R/W = 1
ACK	ACK
Data1 / ACK	Data1 / ACK
Data2 / ACK	Data2 / ACK
Data3 / ACK	Data3 / NACK
STOP	STOP

8.6 Key Concepts

During Write

- Master controls the transmission
- Slave acknowledges every received byte
- Communication ends with a STOP condition

During Read

- Slave transmits data

- Master acknowledges each received byte
- Master sends NACK after the final byte to indicate the end of reception
- STOP condition follows the NACK

8.7 Interview Questions

Q. How does the slave know that the master wants to write?

Answer: The master sets the R/W bit to 0, indicating a write operation.

Q. How does the slave know that the master wants to read?

Answer: The master sets the R/W bit to 1, indicating a read operation.

Q. Why does the master send ACK during a read operation?

Answer: The ACK informs the slave that the received byte is correct and that the master wants more data.

Q. Why is NACK sent after the last byte in a read operation?

Answer: The NACK tells the slave that the master has finished receiving data and no additional bytes should be transmitted.

Q. Who generates the STOP condition?

Answer: The master always generates the STOP condition to terminate the communication.

8.8 Memory Trick

■ *Master WRITE (R/W=0): Master sends Address, Master sends Data, Slave sends ACK, Master sends STOP. Think: 'Master Writes, Slave Acknowledges.'*

■ *Master READ (R/W=1): Master sends Address, Slave sends Data, Master sends ACK after each byte, Master sends NACK after the last byte, Master sends STOP. Think: 'Slave Sends, Master Acknowledges Until Done.'*

8.9 Exam & Revision Points

- Communication always begins with a START condition
- The 7-bit slave address is followed by the R/W bit
- R/W = 0 → Write operation; R/W = 1 → Read operation
- During a write, the master sends data and the slave ACKs each byte
- During a read, the slave sends data and the master ACKs each byte except the last
- The master sends NACK after the final byte to stop the slave from transmitting further
- The communication always ends with a STOP condition generated by the master

9. Repeated START (Sr)

Definition: A Repeated START, denoted by **Sr**, is a START condition generated without first sending a STOP condition.

■ *Repeated START = START again without STOP.*

Instead of ending the communication with a STOP and then beginning a new one with another START, the master directly issues another START condition while still controlling the bus.

9.1 Why is Repeated START Needed?

In many I²C transactions, the master must perform multiple operations as one continuous transaction, such as Write → Read or Read → Write.

If the master generates a STOP between these operations, it releases the bus. This allows another master (in a multi-master system) to take control before the original transaction is complete. A Repeated START prevents this by keeping control of the bus until the entire transaction finishes.

9.2 What Problem Does It Solve? (EEPROM Example)

Consider: a master microcontroller and an EEPROM connected as an I²C slave. The master wants to read the data stored at memory address 0x45 inside the EEPROM.

The EEPROM does not know which memory location the master wants to read. Therefore, the master must first tell the EEPROM the memory address. This requires two operations:

- Write the memory address (0x45)
- Read the data stored at that address

So the communication becomes: WRITE Address → READ Data

9.3 Method 1 — Without Repeated START

Communication Sequence

- Step 1: Master generates START
- Step 2: Master sends Slave Address, R/W = 0 (Write)
- Step 3: EEPROM sends ACK
- Step 4: Master writes 0x45 (this is not data — it is the memory address to read from later)
- Step 5: EEPROM acknowledges and prepares the contents stored at address 0x45
- Step 6: Master generates STOP — communication ends
- Step 7: Master starts another communication: START
- Step 8: Master sends Slave Address, R/W = 1 (Read)
- Step 9: EEPROM sends data stored at address 0x45
- Step 10: Master finishes communication with STOP

Timing Sequence (Without Repeated START)

```
START
   |
Address + Write (R/W=0)
   |
ACK
   |
Memory Address = 0x45
   |
ACK
   |
STOP
```

```
------------------------------
START
   |
Address + Read (R/W=1)
   |
ACK
   |
Data at 0x45
   |
NACK
   |
STOP
```

Problem With This Method

Notice what happens after the first STOP:

```
START
 v
Write Address
 v
STOP   <- Bus Released
```

■ **GOLDEN RULE: The master has not completed its actual task (reading the EEPROM), yet the STOP releases the bus. If another master exists, it may take control before the first master begins its read operation — interrupting the intended transaction.**

9.4 Method 2 — Using Repeated START

Instead of sending STOP, the master sends another START — this is a **Repeated START (Sr)**.

Communication Sequence

- Step 1: Master generates START
- Step 2: Master sends Slave Address, R/W = 0
- Step 3: EEPROM ACKs
- Step 4: Master writes 0x45 (memory address)
- Step 5: EEPROM ACKs
- Step 6: Instead of STOP, master generates Repeated START (Sr) — the bus is not released
- Step 7: Master again sends Slave Address, R/W = 1 (indicating a read operation)
- Step 8: EEPROM sends the data stored at address 0x45
- Step 9: Master sends NACK after receiving the last byte
- Step 10: Master finally generates STOP — communication ends

Timing Sequence (Using Repeated START)

```
START
   |
Address + Write (R/W=0)
   |
ACK
   |
Memory Address = 0x45
   |
ACK
   |
Repeated START (Sr)
   |
Address + Read (R/W=1)
   |
ACK
   |
```

```
Data at 0x45
   |
NACK
   |
STOP
```

9.5 Key Observation

Without Repeated START	With Repeated START
Write → STOP → Read	Write → Repeated START → Read → STOP
Bus becomes free between operations	Bus remains occupied by the same master until the transaction completes

9.6 Bus Ownership

Without Repeated START	With Repeated START
Master → Write → STOP → Bus becomes FREE. Another master may acquire the bus.	Master → Write → Repeated START → Read → STOP. Bus released only after completion.

9.7 Advantages of Repeated START

- Prevents releasing the bus between related operations
- Ensures atomic (uninterrupted) transactions
- Prevents another master from interrupting communication
- Ideal for devices like EEPROMs and sensors that require a register or memory address to be written before reading

9.8 When Should Repeated START Be Used?

Commonly used when performing Write → Read or Read → Write, for example:

- Reading data from an EEPROM
- Reading a sensor register (write register address, then read data)
- Accessing configuration registers of peripherals

9.9 Multi-Master vs Single-Master Bus

Bus Type	Recommendation
Multi-Master Bus	Repeated START is highly recommended — it prevents another master from taking control of the bus before the transaction is complete.
Single-Master Bus	Since there is only one master, no other device can claim the bus. Either STOP+START or Repeated START works correctly. Repeated START is still commonly used because many I²C devices expect this sequence.

9.10 Comparison Summary

Without Repeated START	With Repeated START
STOP generated between operations	No STOP between operations
Bus is released	Bus remains occupied
Another master may take control	No other master can interrupt

Two separate transactions	One continuous transaction
Suitable for single-master systems	Preferred for multi-master systems and register-based devices

9.11 Complete Transaction Comparison

```
WITHOUT REPEATED START              WITH REPEATED START

START                                START
Address + Write                      Address + Write
ACK                                  ACK
Memory Address                       Memory Address
ACK                                  ACK
STOP                                 Repeated START
                                      Address + Read
START                                ACK
Address + Read                       Data
ACK                                  NACK
Data                                 STOP
NACK
STOP
```

9.12 Interview Questions

Q. What is a Repeated START in I²C?

Answer: A Repeated START is a START condition issued without a preceding STOP condition. It allows the master to continue communication without releasing the bus.

Q. Why is Repeated START used?

Answer: It prevents the bus from being released between related operations, ensuring another master cannot interrupt the transaction.

Q. When is Repeated START commonly used?

Answer: During combined operations such as Write → Read or Read → Write, especially when reading registers or memory locations from devices like EEPROMs and sensors.

Q. Why does an EEPROM require a Write before a Read?

Answer: The master must first send the memory address (for example, 0x45) so the EEPROM knows which location's data should be returned during the subsequent read operation.

Q. Is Repeated START necessary in a single-master system?

Answer: Not strictly. Since no other master can access the bus, either STOP + START or Repeated START works. However, Repeated START is still widely used because many I²C devices are designed to operate with this sequence.

9.13 Memory Trick

■ *"STOP releases the bus, Repeated START keeps the bus." Or remember: START → Write → Sr → Read → STOP*

10. I²C Driver Development — STM32F407x Peripheral Overview

After understanding the I²C protocol (START, STOP, ACK/NACK, Read/Write, and Repeated START), the next step is to develop an I²C driver for the microcontroller. Driver development begins by studying the I²C peripheral hardware available in the microcontroller.

10.1 Step 1: Identify the I²C Peripherals

Before writing a driver:

- Open the microcontroller's Reference Manual
- Find the I²C peripheral section
- Determine the number of I²C peripherals available, the bus to which they are connected, and the registers associated with each peripheral

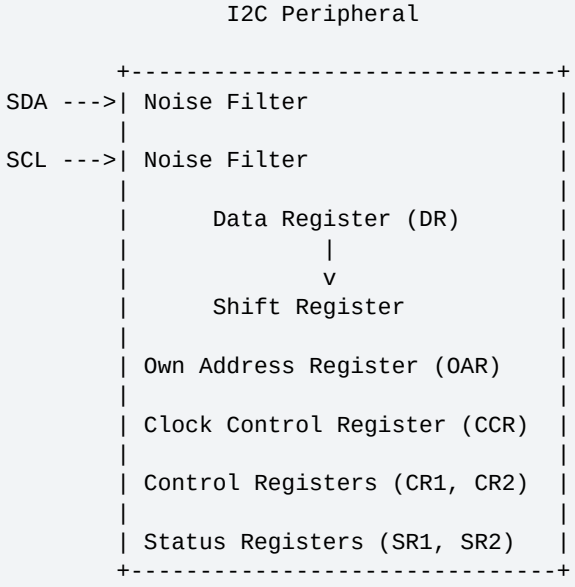
Example: STM32F407x

Peripheral	Connected Bus
I²C1	APB1
I²C2	APB1
I²C3	APB1

Observation: Total I²C peripherals = 3, all connected to the APB1 (Advanced Peripheral Bus 1).

■ *Other STM32 microcontrollers may have only 2, 3, or even more I²C peripherals. Always verify this from your device's reference manual.*

10.2 I²C Peripheral Block Diagram



10.3 Major Components of the I²C Peripheral

1. SDA and SCL Pins

Pin	Full Form	Purpose
SDA	Serial Data	Transfers data
SCL	Serial Clock	Synchronizes communication

■ *Although STM32 I²C peripherals also support the SMBus (System Management Bus) protocol, it is not covered in this course.*

2. Noise Filter

Before signals reach the internal logic, both SDA and SCL pass through a noise filter, which removes glitches, spikes, short unwanted pulses, and electrical noise, improving signal reliability.

```
SDA -----> Noise Filter -----> Internal Logic

SCL -----> Noise Filter -----> Internal Logic
```

- Improves communication reliability
- Prevents false START or STOP detection
- Filters out high-frequency noise

3. Data Register (DR)

The I²C peripheral contains only **one** Data Register (DR). Why only one? I²C is a half-duplex communication protocol — transmission and reception do not occur simultaneously, so separate TX and RX data registers are unnecessary.

During Transmission:

```
CPU
 |
 v
Data Register (DR)
 |
 v
Shift Register
 |
 v
SDA Pin
```

- CPU writes one byte into DR
- Hardware copies it to the Shift Register
- Shift Register sends bits serially through SDA

During Reception:

```
SDA
 |
 v
Shift Register
 |
 v
Data Register (DR)
 |
 v
CPU Reads
```

- Serial bits arrive through SDA
- Shift Register collects 8 bits
- After a complete byte is received, it is copied into DR
- Firmware reads the byte from DR

4. Shift Register

The Shift Register performs serial communication:

- During Transmission: converts 8-bit Parallel Data → Serial Bit Stream
- During Reception: converts Serial Bit Stream → 8-bit Parallel Data

It acts as the interface between the internal data register and the external SDA line.

5. Own Address Register (OAR)

The Own Address Register (OAR) stores the slave address of the microcontroller — used only in **Slave Mode**. If the microcontroller acts as an I²C slave, its slave address must be stored in OAR. When another master sends this address, the peripheral recognizes it is being addressed and responds.

Example: Slave Address = 0x52 → Stored in OAR.

■ *Not required in Master Mode: when the microcontroller acts as a master, it initiates communication and does not need its own slave address.*

6. Clock Control Block

The SCL line is connected internally to the Clock Control Block, which generates the clock signal used during communication.

7. Clock Control Register (CCR)

The Clock Control Block is configured using the Clock Control Register (CCR), which determines the frequency of the SCL signal (e.g., 100 kHz Standard Mode, 400 kHz Fast Mode). The firmware configures CCR to generate the desired I²C clock speed. (CCR is studied in detail during driver implementation.)

8. Control Registers

Register	Purpose
CR1	Enables/disables the peripheral and controls important operating features
CR2	Configures additional operating parameters such as clock frequency and interrupts

9. Status Registers

Register	Purpose
SR1	Indicates communication events and status flags
SR2	Provides additional status information

Examples of status information include: START condition generated, address matched, data transmitted, data received, ACK received, bus busy. The driver continuously checks these status flags to determine the progress of communication.

10.4 Data Flow Inside the I²C Peripheral

Transmit Operation

```
CPU
 |
 v
Data Register (DR)
 |
 v
Shift Register
 |
 v
SDA Pin
 |
 v
Slave Device
```

Receive Operation

```
Slave Device
 |
 v
```

```
SDA Pin
 |
 v
Shift Register
 |
 v
Data Register (DR)
 |
 v
CPU
```

10.5 Master Mode vs Slave Mode

Feature	Master Mode	Slave Mode
Initiates communication	Yes	No
Generates SCL	Yes	No
Needs Own Address Register	No	Yes
Uses CCR to generate clock	Yes	No (clock supplied by master)

10.6 Registers Introduced — Summary

Register	Full Form	Function
DR	Data Register	Stores transmitted or received data
OAR	Own Address Register	Stores the device's slave address
CCR	Clock Control Register	Configures SCL frequency
CR1	Control Register 1	Enables and configures the peripheral
CR2	Control Register 2	Additional configuration options
SR1	Status Register 1	Communication status flags
SR2	Status Register 2	Additional communication status

10.7 Key Takeaways

- Driver development begins by understanding the microcontroller's I²C hardware
- The STM32F407x has three I²C peripherals (I²C1, I²C2, I²C3) connected to the APB1 bus
- Communication uses only two external lines: SDA and SCL
- Both lines pass through noise filters to suppress glitches
- A single Data Register (DR) is used because I²C is half-duplex
- The Shift Register handles serial transmission and reception
- The Own Address Register (OAR) is required only when the device operates as an I²C slave
- The Clock Control Register (CCR) determines the SCL clock frequency
- CR1 and CR2 configure the peripheral
- SR1 and SR2 report communication status and events

10.8 Interview Questions

Q. How many I²C peripherals are available in the STM32F407x?

Answer: Three: I²C1, I²C2, and I²C3, all connected to the APB1 bus.

Q. Why does the I²C peripheral have only one Data Register?

Answer: Because I²C is a half-duplex protocol. Transmission and reception occur one at a time, so separate transmit and receive data registers are unnecessary.

Q. What is the function of the Own Address Register (OAR)?

Answer: It stores the microcontroller's slave address and is used only when the device operates in slave mode.

Q. What is the purpose of the Clock Control Register (CCR)?

Answer: The CCR configures the SCL clock frequency, allowing the I²C peripheral to operate at different speeds such as 100 kHz or 400 kHz.

Q. Why are noise filters placed on SDA and SCL?

Answer: To remove glitches, spikes, and electrical noise, improving communication reliability and preventing false signal detection.

10.9 Memory Trick

■ *Register order: DR → OAR → CCR → CR1/CR2 → SR1/SR2. (DR – Data, OAR – Own Address, CCR – Clock Control, CR – Configure, SR – Status)*

10.10 Exam & Revision Points

- Check the reference manual to identify the available I²C peripherals
- STM32F407x contains three I²C peripherals on APB1
- The two primary I²C pins are SDA and SCL
- Noise filters improve signal quality
- I²C uses one Data Register because it is half-duplex
- The Shift Register converts between parallel and serial data
- OAR stores the slave address in slave mode
- CCR controls the SCL frequency
- CR1/CR2 configure the peripheral
- SR1/SR2 indicate communication status

■ *One-Line Revision: The STM32 I²C peripheral consists of SDA/SCL pins with noise filters, a single Data Register and Shift Register for half-duplex communication, an Own Address Register for slave mode, a Clock Control Register for SCL generation, Control Registers (CR1/CR2) for configuration, and Status Registers (SR1/SR2) for monitoring communication events.*

— End of Notes —